RookDB System Documentation

Team 1: George, Mithun, Sujay

March 27, 2026

1 Project Overview

RookDB is a disk-oriented relational database management system built from the ground up. Initially, RookDB relied entirely on generic sequential table scans, which produced increasingly slow performance as dataset size grew. To address this limitation, a generalized and highly versatile Indexing Subsystem has been integrated between the Storage layer and the Catalog layer.

This documentation covers the complete system architecture, internal data structures, standard operational workflows, and implementation details across all RookDB components, with a heavy emphasis on the Indexing Subsystem introduced in the original design and proposal phases.

1.1 Key System Assumptions and Limitations

Constraint	Details
Execution Model	Single-threaded; no native concurrency control or locking on tables or indexes.
Transactions	Operations represent immediate persistence; no ACID rollbacks or Write-Ahead Logging (WAL).
Deletions	Tuples are marked deleted in-place (length = 0) and removed from all indexes. Space is not reclaimed without periodic rebuilding.
Supported Types	`INT`, `FLOAT`, and `TEXT` — all natively hashable and comparable as index keys.

Table 1: RookDB System Constraints

Note on Durability: Because there is no WAL or transaction rollback mechanism, system crashes mid-operation may leave the database in a partially written state.

2 Global Architecture and Storage Engine

RookDB organizes memory and persistent storage cleanly across several discrete subsystems, each with a well-defined responsibility boundary.

2.1 Disk and Page Management

`src/backend/disk/` `src/backend/page/`

2.1.1 Fixed Page Sizes

Both table data files and index files strictly use **8 KB (8,192 byte)** pages. This uniform page size simplifies I/O and buffer management significantly.

2.1.2 Slotted Page Architecture

Pages follow a **slotted page design**, enabling graceful handling of variable-length column data.

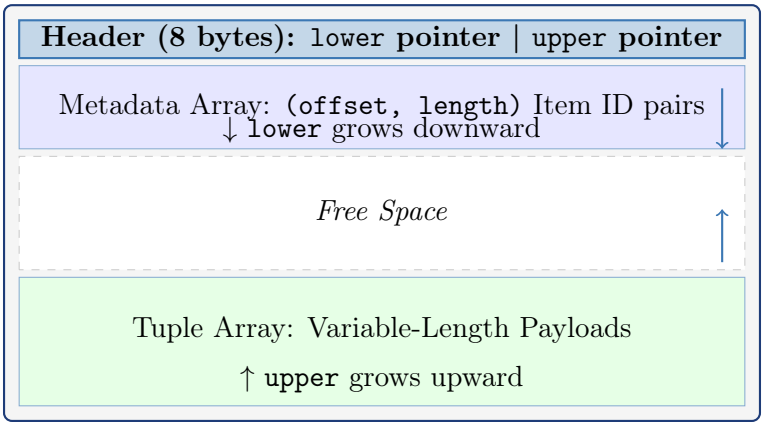

Figure 1: Slotted Page Layout — 8 KB Fixed-Size Page

The key structural elements of each page are:

- **Header (8 bytes):** Stores the `lower` pointer (where free space begins) and the `upper` pointer (where tuple records begin from the page end).
- **Metadata Array:** Located at the top of the page, progressing downward. Each entry is a pair `(offset, length)` representing a physical *Item ID*.
- **Tuple Array:** Located at the bottom of the page, progressing upward. Tuple payloads physically reside here as raw bytes.

2.2 Buffer Manager

`src/backend/buffer_manager/`

The Buffer Manager is an in-memory cache system that groups vectors of `Page` structs. Its key behaviors include:

- During bulk operations (such as `load_csv_into_pages`), the manager **aggressively caches** all newly formatted binary slotted pages into available `Vec<Page>` buffers.
- Pages are periodically flushed to disk via `flush_to_disk()`.
- This design minimizes redundant disk I/O during heavy batch load operations.

2.3 Heap Manager

`src/backend/heap/mod.rs`

The Heap Manager governs standard table state, handling insertions and mutations at the individual tuple level.

- **Dynamic space management:** Checks available page space via `page_free_space`; requests additional disk page allocations when the existing block exceeds the 8 KB limit.
- **Index propagation:** Transparently propagates all changes to the Indexing tier via wrapper methods such as `insert_tuple_with_index_maintenance`.
- **Record Identification:** Each tuple's definitive physical location is encoded as:

$$\texttt{RecordId} \triangleq \{\, \texttt{page_no:}\ \ \texttt{u32},\ \texttt{item_id:}\ \ \texttt{u32} \,\}$$

3 Catalog and Metadata Management

`src/backend/catalog/`

The database layout tracks schemas using a **flat JSON hierarchy** stored in `CATALOG_FILE` (`catalog.json`).

3.1 Catalog Hierarchy

$$\textbf{Catalog} \longrightarrow \textbf{Database} \longrightarrow \textbf{Table}$$

Each **Table** in the catalog maintains:

- `columns` — An array of `Column` structures, each detailing a `name` and `data_type`.
- `indexes` — A collection of `IndexEntry` metadata objects, containing algorithm type, covered column(s), clustered flag, and configuration parameters.

Consistency Enforcement: Before any disk creation or fetch operation, instances are validated against the parsed catalog tree, checking bounds and uniqueness. Table data files are located at:

`database/base/{db_name}/{table_name}.dat`

4 The Indexing Subsystem

`src/backend/index/`

The Index abstraction operates through a unified `IndexTrait` interface. All index implementations must provide the following operations:

Method	Description
`insert(...)`	Insert a `(key, RecordId)` mapping into the index.
`search(&key)`	Retrieve all `RecordId`s matching a given key.
`delete(&key)`	Remove a key and its associated records from the index.
`entry_count()`	Return the current number of entries stored.
`save(...)`	Persist the index structure to disk as a `.idx` file.

Table 2: `IndexTrait` Required Interface

4.1 Type Dispatcher: `AnyIndex`

Because Rust's trait objects (`Box<dyn IndexTrait>`) lack static constructors for deserializing from disk, the system uses a type-erased enum struct called `AnyIndex`. This wrapper safely relays trait operations to up to **9 distinct data structures**.

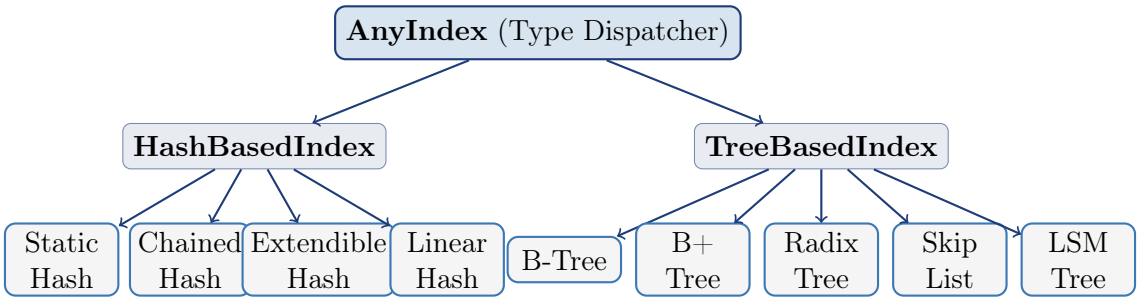

Figure 2: `AnyIndex` Dispatcher and Supported Index Structures

4.2 Hash-Based Indexes (`HashBasedIndex`)

Hash-based indexes optimize **exact-match point queries** to $O(1)$ average-case complexity.

4.2.1 1. Static Hash Index

A fixed-size bucket implementation suitable for datasets with known cardinality and bounded overflow. Bucket count is set at creation and does not grow.

4.2.2 2. Chained Hash Index

Upon collision, overflowing keys are appended to a **linked chain of overflow buffers** external to the primary bucket array. Simple to implement; degrades when chains grow long.

4.2.3 3. Extendible Hash Index

A dynamic hash table using **directory doubling** instead of full table rehashing.

- Maintains a *directory* of pointers targeting physical buckets.
- Each bucket tracks a `local_depth`; the directory tracks a `global_depth`.
- On overflow, only the affected bucket is split and its `local_depth` is incremented.
- The directory doubles in size only when `local_depth == global_depth`, avoiding unnecessary full rehashes.

4.2.4 4. Linear Hash Index

Uses a managed **round-robin bucket splitting strategy** via an internal `split_pointer`.

- Splits are triggered incrementally as the load factor exceeds a threshold.
- Prevents the sudden $O(N)$ doubling penalty inherent in directory-based schemes.
- Ideal for write-heavy workloads requiring smooth, predictable performance.

4.3 Tree-Based Indexes (`TreeBasedIndex`)

Tree-based indexes maintain strict **sorted order**, enabling both $O(\log n)$ point lookups and efficient **range scans**.

4.3.1 5. B-Tree

A classic **multi-level balanced tree**. Both inner nodes and leaf nodes store keys and record pointers. Efficient for general-purpose sorted access, but range scans may require traversing back up to ancestor nodes.

4.3.2 6. B+ Tree

The conventional standard for database index structures. Key design properties:

- **All valid record pointers** are stored exclusively in leaf nodes.
- Inner nodes store only key routing information.
- Leaf nodes maintain a `next_leaf` linked-list pointer, enabling dense, contiguous range scans.
- Range scan complexity: $O(\log n + k)$ where k is the number of matching records.
- This is RookDB's **default index structure** for Primary Key indexes.

4.3.3 7. Radix Tree

A **bit-wise prefix trie** representation that scales particularly well for string key data.

- Compresses shared key prefixes into compact internal nodes.
- Efficient for prefix-range queries on `TEXT` columns.

4.3.4 8. Skip List

A **probabilistic multi-layer linked list** hierarchy allowing efficient ordered access.

- Index probing skips large portions of the list via express lanes at higher levels.
- Well-suited for workloads with frequent insertions and deletions due to low rebalancing cost.
- Expected $O(\log n)$ search, insert, and delete.

4.3.5 9. LSM Tree (Log-Structured Merge Tree)

Optimized heavily for **write-intensive workloads** using immutable layered generation tiers.

- Writes are initially buffered in-memory, then flushed as immutable sorted runs.
- Background compaction merges runs across tiers, amortizing write amplification.
- Delivers excellent sequential write throughput at the cost of slightly higher read latency.

Structure	Point Query	Range Scan	Best Use Case
Static Hash	$O(1)$	$\times$	Fixed-cardinality exact match
Chained Hash	$O(1)$ avg	$\times$	General exact match
Extendible Hash	$O(1)$ avg	$\times$	Dynamic growing workloads
Linear Hash	$O(1)$ avg	$\times$	Smooth write throughput
B-Tree	$O(\log n)$	$O(\log n + k)$	General sorted access
B+ Tree	$O(\log n)$	$O(\log n + k)$	DB primary keys, range queries
Radix Tree	$O(\|key\|)$	$O(\log n + k)$	String/prefix queries
Skip List	$O(\log n)$	$O(\log n + k)$	Frequent updates
LSM Tree	$O(\log n)$	$O(\log n + k)$	Write-heavy workloads

Table 3: Index Structure Performance Comparison

4.4 Index Categories

RookDB supports three standard SQL-level index categories:

4.4.1 Primary Key Index

- Uniquely maps identities — duplicate key insertions are rejected with an error.
- Defaults to a B+ Tree structure.
- Enforced via strict collision checks during all tuple mutations.

4.4.2 Clustered Index

- Defines the **physical read ordering** of the base table.
- When scanning via a Clustered B+ index, RookDB accumulates all matching `RecordId`s and **sorts them** before fetching from disk (`index_scan.rs`). This dramatically improves page cache locality without destructively reorganizing the underlying `.dat` file.
- **Only one Clustered index** may exist per table.

4.4.3 Secondary (Covering) Index

- Provides standalone sorted access per column.
- Multiple composite secondary indexes are supported.
- Accepts an `{ include_columns }` configuration parameter — when the queried columns are entirely contained within the index node, RookDB can satisfy the query **without touching the parent table**, eliminating secondary page fetches.

5 Executor Engine and Scans

`src/backend/executor/`

During generalized table searches (`SHOW TUPLES` / `SEARCH`), the executor first queries the `Catalog` to determine if an applicable `IndexEntry` exists for the requested parameter.

5.1 Sequential Scan

`seq_scan.rs`

Legacy fallback behavior. Iterates continuously page by page across the entire table file:

- Deserializes each page's header `lower` pointer incrementally.
- Extracts memory boundary pairs using `u32::from_le_bytes`.
- Evaluates every tuple in order — no skipping is possible.
- Complexity: $O(N)$ where N is the total number of tuples.

Performance: Sequential scans are the fallback path when no applicable index exists. They are functional but significantly slower than index scans on large datasets.

5.2 Index Scan

`index_scan.rs`

Direct table probing using a pre-built index structure. The procedure is:

1. Look up the `.idx` file corresponding to the requested column using the `Catalog` configuration.
2. Construct an `AnyIndex` object in memory from the persisted payload via `AnyIndex::load()`.
3. Call `.search(&key)`, which natively returns a `Vec<RecordId>`.
4. For Clustered indexes, sort the `RecordId` list before disk access to maximize cache locality.
5. Fetch the corresponding 8 KB physical page blocks and return the matching tuples.

6 Standard Workflows

6.1 Index Creation

1. User provides index definitions: `INDEX TYPE`, `CLUSTERED` flag, and `COLUMN_NAME`.
2. The database validates that the referenced column exists in the Table `catalog`.
3. Validates that no pre-existing clustered index conflict exists.
4. Generates an `AnyIndex` node using the matching backend data structure configuration.
5. Loads all target table pages block by block, reading tuples into byte chunks, and parsing the target property according to its data type (`TEXT / INT / FLOAT`).
6. Iteratively inserts `(IndexKey, RecordId)` pairs into the initialized structure via `insert()`.
7. On completion: updates `catalog.json` with the new index entry and saves the `.idx` file to disk.

6.2 Table Modification (Insert / Delete)

All heap tuple mutations bind directly through `insert_tuple_with_index_maintenance`:

Operation	Behavior
Insert	Extracts payload values for all indexes recorded in the Catalog, maps them into memory, then saves the updated index pages to disk.
Delete	Validates record existence. Marks the page metadata element's length as zero (in-place deletion marker). Signals all registered indexes via `remove_tuple_from_all_indexes`, wiping dangling `(page, item)` identifier entries.

Table 4: Index Maintenance During Table Mutations

6.3 Standard Searching (Point Queries)

Frontend requests parse query tokens into logical `IndexKey` instances and pass them to `.search()`:

- If a viable index is found in the parsed Catalog configuration, an **Index Scan** is triggered.
- If no index is applicable, a fallback **Sequential Scan** is performed, ensuring functional (though slower) results.

6.4 Range Scans (Tree Indexes Only)

For queries matching predicates of the form `START` $\leq$ `key` $\leq$ `END`:

- Range scans are supported **exclusively** by `TreeBasedIndex` types.
- The backend traverses internal tree nodes to locate the leaf containing the lowest key $\geq$ `START`.
- Iterates across chained leaf references (e.g., `next_leaf` pointers in B+ Trees), collecting all matching `RecordId`s until the `END` predicate is breached.
- This yields $O(\log n + k)$ range scan performance.

Hash Index Limitation: Hash-based indexes *cannot* support range scans. Only tree-based structures maintain the sorted order required for bounded range traversal.

7 Frontend Menu and CLI

`src/frontend/menu.rs`

The command-line interface (CLI) maps user-facing commands to the appropriate backend disk and index operations:

- **Table and Database Creation:** Resolves creation commands to disk operations, creating catalog entries and base `.dat` files.
- **Data Loading:** Implements bulk data ingestion hooks via rapid buffered interactions (e.g., `Synthetic Orders CSV Benchmarking`), allowing large-scale dataset generation.
- **Benchmarking:** Configures benchmark routines that measure exact millisecond differences between index-accelerated scans and sequential scans, returning comparative performance logs.

8 Summary: Directory Structure Reference

Path	Responsibility
`src/backend/disk/`	Raw disk I/O, page file management
`src/backend/page/`	Slotted page layout, header encoding
`src/backend/buffer_manager/`	In-memory page cache, flush to disk
`src/backend/heap/mod.rs`	Tuple insertions, deletions, space management
`src/backend/catalog/`	JSON schema catalog, IndexEntry metadata
`src/backend/index/`	IndexTrait, AnyIndex dispatcher, all 9 structures
`src/backend/executor/`	Sequential scan, index scan, query routing
`src/frontend/menu.rs`	CLI command parsing, benchmarking
`database/base/{db}/{tbl}.dat`	Physical table data files
`catalog.json`	Global schema and index configuration

Table 5: RookDB Source Directory Reference